THE SEIFERT-VAN KAMPEN THEOREM VIA COMPUTATIONAL PATHS: A FORMALIZED APPROACH TO COMPUTING FUNDAMENTAL GROUPS

ARTHUR F. RAMOS, TIAGO M. L. DE VERAS, RUY J. G. B. DE QUEIROZ, AND ANJOLINA G. DE OLIVEIRA

Microsoft, USA
e-mail address: arfreita@microsoft.com

Departamento de Matemática, Universidade Federal Rural de Pernambuco, Brazil
e-mail address: tiago.veras@ufrpe.br

Centro de Informática, Universidade Federal de Pernambuco, Brazil
e-mail address: ruy@cin.ufpe.br

Centro de Informática, Universidade Federal de Pernambuco, Brazil
e-mail address: ago@cin.ufpe.br

Abstract. The Seifert-van Kampen theorem computes the fundamental group of a space from the fundamental groups of its constituents. We develop a *modular SVK framework* within the setting of *computational paths*—an approach to equality where witnesses are explicit sequences of rewrites governed by the $\mathrm{LND}_{\mathrm{EQ}}$-TRS.

Our contributions are: (i) pushouts as quotient types with explicit glue paths; (ii) free and amalgamated products as word quotients; (iii) a proof-relevant notion of *presented computational path space*, whose raw paths are freely generated before any normalization and whose homotopies are generated by named relators and the groupoid laws; and (iv) a proved Seifert–van Kampen equivalence

$$\pi_1^{\mathrm{pres}}(\mathcal{P}(G_1, G_2; H), *) \cong G_1 *_H G_2.$$

The Lean theorem `presentedSeifertVanKampenGroupEquiv` constructs both maps, proves both round trips, and records preservation of multiplication, identity, and inversion. Separately, the presented circle has fundamental group $\mathbb{Z}$. The figure-eight specialization of SVK is the full free product of two copies of $\mathbb{Z}$, with a separately proved nontrivial generator.

The distinction between this presented path groupoid and the library's global `PathRwQuot` is explicit. Function-space rules make every loop fiber of the latter contractible; consequently, its SVK theorem is degenerate unless encode/decode data are assumed. Scoped completions remain available for the torus, Klein bottle, projective plane, and cyclic lens-space expression languages, but they are not identified with the global quotient or with ordinary topological fundamental groups.

The nineteen Lean modules in the formalization core described here comprise **11,796 lines**. The complete repository contains no `sorry`, `admit`, or custom `axiom` declaration. Source, build instructions, and the reproducibility script are available at `https://github.com/Arthur742Ramos/ComputationalPathsLean`.

Key words and phrases: Seifert-van Kampen theorem, computational paths, fundamental group, higher-inductive types, pushouts, free products, type theory.

1. Introduction

The Seifert-van Kampen theorem, first proved by Herbert Seifert [18] and Egbert van Kampen [20], is one of the most powerful tools in algebraic topology for computing fundamental groups (see [7, 13] for modern treatments). It states that if a path-connected space X is the union of two path-connected open subspaces U and V with path-connected intersection $U \cap V$, then the fundamental group $\pi_1(X)$ can be computed as the amalgamated free product:

$$\pi_1(X) \cong \pi_1(U) *_{\pi_1(U \cap V)} \pi_1(V)$$

In homotopy type theory (HoTT) [19], this theorem takes a particularly elegant form when expressed in terms of *pushouts*—a higher-inductive type (HIT) that generalizes the notion of gluing spaces together. The HoTT version of the Seifert-van Kampen theorem was developed by Favonia and Shulman [8], providing a synthetic proof that works entirely within type theory.

The present paper develops a *modular SVK framework* within the setting of *computational paths* [2, 14]—an alternative approach to equality in type theory where witnesses of equality are explicit sequences of rewrites. This approach offers several advantages:

(1) **Explicit witnesses**: Every equality proof carries the full computational content showing *why* two terms are equal.
(2) **Syntactic path equality**: The rewrite equality relation ($\sim$) is the equivalence closure generated by the `Step` rules. Termination and confluence are mechanized for the abstract groupoid expression fragment. For arbitrary trace-carrying paths, normalization is a sound invariant, not a complete characterization: decidability is available only after supplying a `CompleteStepSystem` (Section 2.4).
(3) **Concrete coherence**: Higher coherence witnesses (pentagon, triangle, etc.) are explicit rewrite derivations rather than abstract existence proofs.
(4) **Modular assumptions**: The SVK framework uses *split typeclass assumptions*, allowing users to provide only what they need—from full encode/decode data to minimal Prop-level bijectivity claims.

Relation to Classical SVK.. The classical theorem requires $X = U \cup V$ with $U, V, U \cap V$ open and path-connected. The HIT version generalizes this: the pushout $\mathrm{Pushout}(A, B, C, f, g)$ corresponds to the union where f and g specify how the "intersection" C embeds into both spaces. Path-connectedness assumptions are needed to ensure the fundamental group is well-defined; in the HIT setting, this becomes the assumption that all types are path-connected.

1.1. Main Contributions.

This paper provides:

(1) Pushouts implemented via Lean's quotient types (*zero kernel axioms*), with *modular typeclass assumptions* for computation rules:
   - Point constructors $\mathrm{inl} : A \to \mathrm{Pushout}$ and $\mathrm{inr} : B \to \mathrm{Pushout}$ (definitions, not axioms)
   - Path constructor $\mathrm{glue} : \prod_{c:C} \mathrm{Path}(\mathrm{inl}(f(c)), \mathrm{inr}(g(c)))$ (via `Quot.sound`)
   - Glue-naturality hypotheses exposed through `HasGlueNaturalLoopRwEq`
   - Proved naturality instances for wedge sums, constant span maps, subsingletons, and explicitly supplied Axiom K data
(2) The construction of free products $G_1 * G_2$ and amalgamated free products $G_1 *_H G_2$, including:

- Amalgamation-only quotient, whose word-length invariant rules out the required SVK round trip
- *Full* amalgamated free product with free group reduction (`FullAmalgamatedFreeProduct`)
- Both data-level and Prop-level (choice-based) interfaces

(3) A proof-relevant path-presentation framework:
- generator graphs and freely generated indexed raw paths;
- homotopy generated from named relators, congruence, and the groupoid laws;
- the fundamental group as the automorphism group of the basepoint in the resulting strict path groupoid.

(4) A proved presented SVK theorem (`presentedSeifertVanKampenGroupEquiv`). Component multiplication, units, inverses, and amalgamation are named relators; encode and decode are constructed afterward and both round trips are proved.

(5) A separate SVK equivalence *schema* for the global `PathRwQuot`, parametric in user-supplied encode/decode structure, with *split typeclass assumptions*:
- `HasPushoutSVKEncodeQuot`: encode map (assumed)
- `HasPushoutSVKDecodeEncode`: decode-encode round-trip (assumed)
- `HasPushoutSVKEncodeDecodeFull`: encode-decode round-trip modulo amalgamation and free-group reduction
- `HasPushoutSVKDecodeFullAmalgBijective`: Prop-level bijectivity alternative
- Decode map and both relation-respecting properties: **proved**

(6) A proved normal-form equivalence for scoped expression completions (`scopedSeifertVanKampenEquiv`), together with its nontrivial figure-eight instance.

(7) An application inventory distinguishing:
- presented computational fundamental groups;
- global-rule `PathRwQuot` results;
- conditional SVK interfaces whose typeclass parameters appear in the theorem statement; and
- scoped expression-presentation quotients.

(8) A Lean 4 development supporting these results, comprising:
- 11,796 lines across the 19 modules used by this paper
- zero `sorry`, `admit`, and custom `axiom` declarations
- a public repository and reproducible source/automation statistics

1.2. **Related Work.** The HoTT–Agda library contains a machine-checked improved van Kampen theorem for the fundamental groupoid of a pushout [9], based on Favonia's formal development. Favonia and Shulman's mechanization of Blakers–Massey provides closely related pushout machinery [8]. These developments use the encode-decode method pioneered by Licata and Shulman [12] for computing $\pi_1(S^1) \cong \mathbb{Z}$.

Lean Mathlib follows a different, classical topological route: its fundamental groupoid has points as objects and continuous paths modulo path homotopy as morphisms, with the fundamental group realized as an automorphism group [10]. The present library instead studies explicit equality traces and their rewrite quotients.

The computational paths approach to equality originates in work by de Queiroz and colleagues [2, 14, 15], building on earlier ideas about equality proofs as sequences of rewrites [3]. Our previous work established that computational paths form a weak groupoid [6] and can be used to calculate fundamental groups [4, 5].

The connection between rewriting and coherence has been developed by Kraus and von Raumer [11]. Their result motivates our completed abstract-expression fragment. We do not infer the converse "equal normalization outputs imply $\sim$" for arbitrary `Path` values without an explicitly supplied complete step system.

1.3. **Reading Guide.** Readers interested in the mathematical content can focus on Sections 3–6, treating Lean code as pseudocode. Those interested in formalization should additionally consult Section 8 and the supplementary code. Key definitions are boxed for quick reference. We use the figure-eight space $S^1 \vee S^1$ as a running example throughout.

1.4. **Code availability.** The complete Lean 4 source is available at `https://github.com/Arthur742Ramos/ComputationalPathsLean`. The paper source, response letter, and reproducible statistics script are in `paper/svk/`. The declarations cited in this article build with the toolchain pinned by `lean-toolchain`; the targeted command is `lake build` followed by the fully qualified module name.

1.5. **Structure of the Paper.** Section 2 reviews computational paths, rewrite equality, and fundamental groups. Section 3 defines quotient pushouts and proves the required glue-naturality consequences. Section 4 constructs the amalgamation-only and full word quotients. Section 5 separates the proved decode construction from the conditional global-rule theorem and the proved scoped theorem. Section 6 gives the application inventory. Section 7 covers extended features. Section 8 discusses the Lean 4 formalization, Section 9 records library highlights, and Section 10 concludes.

2. Background: Computational Paths

2.1. **Computational Paths.** A *computational path* $p : \mathrm{Path}(a, b)$ from a to b (both terms of type A) is an explicit sequence of rewrite steps witnessing the equality $a = b$. The fundamental operations are:

- **Reflexivity**: $\rho(a) : \mathrm{Path}(a, a)$ — the empty sequence of rewrites.
- **Symmetry**: If $p : \mathrm{Path}(a, b)$, then $\sigma(p) : \mathrm{Path}(b, a)$ — reverse and invert each step.
- **Transitivity**: If $p : \mathrm{Path}(a, b)$ and $q : \mathrm{Path}(b, c)$, then $p \cdot q : \mathrm{Path}(a, c)$ — concatenate the sequences.
- **Congruence**: If $p : \mathrm{Path}(a, b)$ and $f : A \to B$, then $f_*(p) : \mathrm{Path}(f(a), f(b))$.
- **Transport**: If $p : \mathrm{Path}(a, b)$ and $P : A \to \mathrm{Type}$, then $\mathrm{transport}(P, p) : P(a) \to P(b)$.

Notation. Throughout this paper, we use:

- $p \cdot q$ for path composition (transitivity)
- p^{-1} for path inverse (symmetry)
- $f_*(p)$ for congrArg(f, p)
- $\rho(a)$ or simply ρ for reflexivity

In Lean, a path is represented as a structure storing both an explicit rewrite trace and an equality proof:

```
structure Path {A : Type u} (a b : A) where
  steps : List (Step A)  -- explicit list of rewrite steps
  proof : a = b          -- the derived equality proof
```

Remark 2.1 (Explicit Step Lists and Consistency). The `steps` field is *the* distinguishing data of a path, not the underlying equality proof. Two paths with the same endpoints may have different step lists, and they are related by $\sim$ only if there is an explicit derivation transforming one step sequence into the other via the Step rules.

This design is critical for consistency: in Lean's proof-irrelevant `Prop`, all equality proofs $p, q : a = b$ satisfy $p = q$ (proof irrelevance). If paths were identified solely by their equality proofs, all loops at a point would be equal, collapsing π_1 to the trivial group.

Instead, path identity depends on the explicit `steps` list. The fundamental group $\pi_1(A, a) := \text{Loop}(A, a)/\sim$ quotients loops by rewrite equivalence of their step sequences—not by equality of their underlying proofs. The $\sim$ relation captures the computational content: two loops are identified only when one step sequence can be transformed to the other via the groupoid laws.

2.2. **The Step Relation.** The foundation of the computational paths framework is the `Step` relation, which defines *single-step rewrites* between paths. The LND_{EQ}-TRS consists of over 50 rewrite rules organized into categories:

- **Groupoid laws**: $\rho^{-1} \rightsquigarrow \rho$, $(p^{-1})^{-1} \rightsquigarrow p$, $\rho \cdot p \rightsquigarrow p$, $p \cdot \rho \rightsquigarrow p$, $p \cdot p^{-1} \rightsquigarrow \rho$, $p^{-1} \cdot p \rightsquigarrow \rho$, $(p \cdot q)^{-1} \rightsquigarrow q^{-1} \cdot p^{-1}$, and $(p \cdot q) \cdot r \rightsquigarrow p \cdot (q \cdot r)$.
- **Type-specific rules**: β-rules for products, sums, and functions; η-rules; transport laws.
- **Context rules**: Allow rewrites inside larger expressions: if $p \rightsquigarrow q$ then $C[p] \rightsquigarrow C[q]$.
- **Congruence closure**: If $p \rightsquigarrow q$ then $p^{-1} \rightsquigarrow q^{-1}$, $p \cdot r \rightsquigarrow q \cdot r$, and $r \cdot p \rightsquigarrow r \cdot q$.

In Lean, the Step relation is defined inductively with 76 constructors:

```
inductive Step : {A : Type u} -> {a b : A} -> Path a b -> Path a
    b -> Prop
  | symm_refl (a : A) : Step (symm (Path.refl a)) (Path.refl a)
  | symm_symm (p : Path a b) : Step (symm (symm p)) p
  | trans_refl_left (p : Path a b) : Step (trans (Path.refl a) p)
      p
  | trans_refl_right (p : Path a b) : Step (trans p (Path.refl b)
    ) p
  | trans_symm (p : Path a b) : Step (trans p (symm p)) (Path.
    refl a)
  | symm_trans (p : Path a b) : Step (trans (symm p) p) (Path.
    refl b)
  | trans_assoc (p : Path a b) (q : Path b c) (r : Path c d) :
```

```
      Step (trans (trans p q) r) (trans p (trans q r))
  | symm_congr : Step p q -> Step (symm p) (symm q)
  | trans_congr_left (r : Path b c) : Step p q -> Step (trans p r
    ) (trans q r)
  | trans_congr_right (p : Path a b) : Step q r -> Step (trans p
    q) (trans p r)
  -- ... plus type-specific rules (products, sums, transport,
    contexts)
```

2.3. **Rewrite Equality.** Two paths $p, q : \mathrm{Path}(a, b)$ are *rewrite equal* ($p \sim q$) if they can be transformed into each other via the $\mathrm{LND}_{\mathrm{EQ}}$-TRS rewrite system. The key rewrite rules include:

$$\rho \cdot p \rightsquigarrow p \qquad \text{(left unit)}$$
$$p \cdot \rho \rightsquigarrow p \qquad \text{(right unit)}$$
$$p \cdot p^{-1} \rightsquigarrow \rho \qquad \text{(right inverse)}$$
$$p^{-1} \cdot p \rightsquigarrow \rho \qquad \text{(left inverse)}$$
$$(p \cdot q) \cdot r \rightsquigarrow p \cdot (q \cdot r) \qquad \text{(associativity)}$$

Example 2.2 (Rewrite Derivation)**.** Consider $p \cdot (q \cdot q^{-1})$ where $p, q : \mathrm{Path}(a, a)$. The derivation to p proceeds:

$$p \cdot (q \cdot q^{-1}) \rightsquigarrow p \cdot \rho \qquad \text{(right inverse)}$$
$$\rightsquigarrow p \qquad \text{(right unit)}$$

This explicit derivation is the "computational content" witnessing that $p \cdot (q \cdot q^{-1}) \sim p$.

In Lean, rewrite equality is defined as the equivalence closure of Step:

```
inductive RwEq {A : Type u} {a b : A} :
    Path a b -> Path a b -> Type (u + 1)
  | refl (p : Path a b) : RwEq p p
  | step {p q : Path a b} : Step p q -> RwEq p q
  | symm {p q : Path a b} : RwEq p q -> RwEq q p
  | trans {p q r : Path a b} : RwEq p q -> RwEq q r -> RwEq p r

abbrev RwEqProp (p q : Path a b) : Prop := Nonempty (RwEq p q)
```

2.4. **Metatheory of the** $\mathrm{LND}_{\mathrm{EQ}}$**-TRS.** The rewrite system $\mathrm{LND}_{\mathrm{EQ}}$-TRS has been studied in prior work [15, 16, 2]. The following table summarizes the status of key metatheoretic properties in this formalization:

Property	**Status**	**Location / responsibility**
Soundness	Proved	`step_toEq` in `Step.lean`: $p \sim q \Rightarrow p.\text{proof} = q.\text{proof}$
Expression termination	Proved	Library instance `instHasTerminationProp`, via a lexicographic weight on `GroupoidTRS.Expr`
Expression confluence	Proved	Library instance `instHasConfluencePropExpr`, via reduced-word interpretation
Path-level confluence	Not globally instantiated	`HasJoinOfRw`/`HasConfluenceProp`; a caller must supply join witnesses because observable step lists obstruct the expression proof
Decidability of $\sim$	Conditional	The caller supplies a `CompleteStepSystem`; then `CompleteStepSystem.decideRwEq` is proved correct

Remark 2.3 (Scope of normalization). The proved expression instances discharge termination and confluence for the completed groupoid-expression subsystem. They are not silently promoted to the trace-carrying `Path` relation. Conversely, `Path.normalize` is invariant under $\sim$, but equality of its outputs does not construct an $\sim$ witness without a `CompleteStepSystem`.

2.5. **Loop Spaces and Fundamental Groups.**

Definition 2.4 (Path-Connected)**.** A type A is *path-connected* if for all $a, b : A$, there exists $p : \text{Path}(a, b)$. Equivalently, $\prod_{a,b:A} \|\text{Path}(a, b)\|_{-1}$ where $\| - \|_{-1}$ denotes propositional truncation.

Definition 2.5 (Loop Space)**.** The *loop space* of a type A at a basepoint $a : A$ is:

$$\Omega(A, a) := \text{Path}(a, a)$$

Definition 2.6 (Global-rule loop quotient)**.** For a Lean type A, the library notation $\pi_1(A, a)$ denotes the quotient of trace-carrying loops by the global rewrite relation:

$$\pi_1(A, a) := \Omega(A, a)/\!\sim$$

In Lean, the fundamental group is realized as a quotient:

```
abbrev LoopSpace (A : Type u) (a : A) : Type u := Path a a

def PiOne (A : Type u) (a : A) : Type u := Quot (@RwEqProp A a a)

notation "pi_1(" A ",␣" a ")" => PiOne A a
```

The group operations on this quotient are induced from path operations:

- Identity: $[\rho]$
- Multiplication: $[\alpha] \cdot [\beta] := [\alpha \cdot \beta]$
- Inverse: $[\alpha]^{-1} := [\alpha^{-1}]$

The group laws (associativity, unit, inverse) hold because the corresponding path identities are witnessed by $\sim$. The function-space rules in this global relation make every such loop fiber contractible (`pathRwQuot_loop_contractible`).

Remark 2.7 (Base Point Dependence). The fundamental group $\pi_1(A, a)$ depends on the choice of basepoint a. For different basepoints $a, b : A$ in a path-connected type, the fundamental groups are isomorphic via conjugation by a path $\gamma : \mathrm{Path}(a, b)$:

$$\pi_1(A, a) \cong \pi_1(A, b), \quad [\alpha] \mapsto [\gamma^{-1} \cdot \alpha \cdot \gamma]$$

However, this isomorphism is not canonical—it depends on the choice of γ.

2.6. **Presented computational path spaces.**

Definition 2.8 (Presented path space)**.** A *presented computational path space* consists of:
(1) a type of points and a proof-relevant family of generating edges;
(2) indexed raw paths freely generated by refl, edge, symm, and trans;
(3) named relators between parallel raw paths.

The generated homotopy relation is the reflexive, symmetric, transitive, congruence closure of the relators and the groupoid laws. Its path classes form a strict groupoid. We define

$$\pi_1^{\mathrm{pres}}(\mathcal{P}, a) := \mathrm{Aut}_{\Pi(\mathcal{P})}(a).$$

This order is important: `Presented.RawPath`, `Presented.Presentation`, and `Presented.Homotopy` are defined before any encode or normalization map. The Lean construction is in `Homotopy/PresentedFundamentalGroup.lean`.

2.7. **Circle path presentation.**

Definition 2.9 (Circle carrier)**.** `CompPath.Circle` is an alias for a one-constructor carrier:
- `circleBase` is its unique point;
- `circleLoop` is a named raw `Path` definition over that point, not a higher-inductive constructor.

Theorem 2.10 (Presented fundamental group of the circle)**.**

$$\pi_1^{\mathrm{pres}}(\mathcal{S}^1_{\mathrm{pres}}, *) \cong \mathbb{Z}.$$

The generator graph has one vertex and integer-labelled loop edges. Its named relators compose labels by addition, remove the zero edge, and identify path reversal with label negation. Winding number is defined recursively on raw paths only after these data. Lean proves that every raw path is homotopic to the edge carrying its winding number; quotient induction gives both encode/decode round trips. The theorem is `CirclePresented.piOneEquivInt`; the lemmas `CirclePresented.encode\_mul`, `CirclePresented.encode\_id`, and `CirclePresented.encode\_inv` prove that it preserves the group operations. Moreover, `CirclePresented.identity_ne_generator` proves that the identity and unit-winding classes differ.

This presented fundamental group is not the global-rule `PathRwQuot Circle circleBase circleBase`, which is contractible. Nor is it asserted to be the ordinary topological fundamental group of a realized circle; that requires a realization/comparison theorem.

Scoped completion semantics. `ScopedCompletion.Data` consists of an expression type E, a normal-form type N, and certified encode/decode maps. Its relation is the equivalence closure of the normalization scheme

$$e \longrightarrow \mathsf{decode}(\mathsf{encode}(e))$$

Thus "scoped" refers to the explicit choice of expression language and normalization rule; no other constructor of the global path-rewrite system is admitted into this quotient. Lean proves `rweq_iff_encode_eq`: two expressions are scoped-rewrite equivalent exactly when their normal forms agree.

As a lighter normal-form interface, `circleScopedPiOneEquivInt` proves

$$\mathsf{CircleScopedPiOne} \simeq \mathbb{Z},$$

and `circleScoped_zero_ne_one` proves that the identity and generator classes are distinct. The product theorem `torusScopedPiOneEquivIntProd` gives $\mathsf{TorusScopedPiOne} \simeq \mathbb{Z}^2$. The same reusable interface is instantiated below for the Klein-bottle, projective-plane, and cyclic lens presentations. The scoped relation excludes unrelated trace-erasing function rules by construction.

3. Quotient Pushouts and Glue Paths

3.1. **The Pushout Type.** Given types A, B, C with maps $f : C \to A$ and $g : C \to B$, the Lean carrier used in this paper is the quotient of $A + B$ generated by $\mathrm{inl}(f(c)) \sim \mathrm{inr}(g(c))$.

$$\begin{array}{ccc} C & \xrightarrow{\quad g \quad} & B \\ {\scriptstyle f}\downarrow & & \downarrow{\scriptstyle \mathrm{inr}} \\ A & \xrightarrow[\mathrm{inl}]{} & \mathrm{Pushout}(A,B,C,f,g) \end{array}$$

Definition 3.1 (Quotient pushout)**.** The quotient pushout $\mathrm{Pushout}(A, B, C, f, g)$ has:

(1) **Point constructors**:

$$\mathrm{inl} : A \to \mathrm{Pushout}(A, B, C, f, g)$$
$$\mathrm{inr} : B \to \mathrm{Pushout}(A, B, C, f, g)$$

(2) **Path constructor**: For each $c : C$,

$$\mathrm{glue}(c) : \mathrm{Path}(\mathrm{inl}(f(c)), \mathrm{inr}(g(c)))$$

In Lean, pushouts are implemented using quotient types (**no kernel axioms**):

```
-- Relation generating the pushout quotient
inductive PushoutCompPathRel (A B C : Type u) (f : C -> A) (g : C
    -> B)
    : Sum A B -> Sum A B -> Prop
  | glue (c : C) :
      PushoutCompPathRel A B C f g (Sum.inl (f c)) (Sum.inr (g c)
          )

-- Pushout as quotient of Sum by PushoutRel
def PushoutCompPath (A B C : Type u) (f : C -> A) (g : C -> B) :
    Type u :=
```

```
  Quot (PushoutCompPathRel A B C f g)

-- Constructors are definitions, not axioms
def inl (a : A) : Pushout A B C f g := Quot.mk _ (Sum.inl a)
def inr (b : B) : Pushout A B C f g := Quot.mk _ (Sum.inr b)
def glue (c : C) : Path (inl (f c)) (inr (g c)) :=
  Path.ofEq (Quot.sound (PushoutCompPathRel.glue c))
```

Remark 3.2 (Quot-based implementation). This is an ordinary quotient, not a native Lean HIT. The point maps and glue path are definitions, and the glue path is constructed via `Quot.sound`. Elimination out of the carrier is ordinary `Quot.lift`.

3.2. **Quotient Recursion.**

Definition 3.3 (Quotient Recursion)**.** Given a type D with:

- $\mathrm{onInl} : A \to D$
- $\mathrm{onInr} : B \to D$
- $h_c : \mathrm{onInl}(f(c)) = \mathrm{onInr}(g(c))$ for every $c : C$

Then `Quot.lift` defines $\mathrm{rec} : \mathrm{Pushout}(A, B, C, f, g) \to D$ with:

$$\mathrm{rec}(\mathrm{inl}(a)) = \mathrm{onInl}(a)$$
$$\mathrm{rec}(\mathrm{inr}(b)) = \mathrm{onInr}(b).$$

3.3. **Modular Typeclass Assumptions.** A key design choice is to expose the exact naturality certificate needed by decode as a typeclass parameter:

```
class HasGlueNaturalLoopRwEq (c0 : C) : Prop where
  eq : forall (c : C) (p : Path c0 c),
    RwEqProp
      (trans (symm (inlPath (congrArg f p)))
        (trans (glue c0) (inrPath (congrArg g p))))
      (glue c)
```

3.4. **Automatic Instance Derivation.** The framework provides proved constructors for common cases:

```
def hasGlueNaturalLoopRwEq_of_axiomK
    (c0 : C) (hC : AxiomK C) :
    HasGlueNaturalLoopRwEq c0 := ...

def hasGlueNaturalLoopRwEq_of_subsingleton
    (c0 : C) [Subsingleton C] :
    HasGlueNaturalLoopRwEq c0 := ...

instance instHasGlueNaturalLoopRwEq_Wedge (a0 : A) (b0 : B) :
    HasGlueNaturalLoopRwEq (C := PUnit')
      (f := fun _ => a0) (g := fun _ => b0) PUnit'.unit := ...
```

These witnesses are implemented in `Pushout.lean` and `PushoutCompPath.lean`. Decidable equality alone is not presented as a discharge mechanism.

3.5. **Glue Naturality.** A key property for the SVK theorem is the *naturality of glue paths*:

Lemma 3.4 (Glue Naturality)**.** *Assume* `HasGlueNaturalLoopRwEq` *at* c_1*. For any path* $p : \mathrm{Path}(c_1, c_2)$ *in* C*, the following square commutes up to* $\sim$*:*

$$\begin{array}{ccc} \mathrm{inl}(f(c_1)) & \xrightarrow{\mathrm{inl}_*(f_*(p))} & \mathrm{inl}(f(c_2)) \\ {\scriptstyle \mathrm{glue}(c_1)}\downarrow & & \downarrow{\scriptstyle \mathrm{glue}(c_2)} \\ \mathrm{inr}(g(c_1)) & \xrightarrow[\mathrm{inr}_*(g_*(p))]{} & \mathrm{inr}(g(c_2)) \end{array}$$

That is, we have the rewrite equality:

$$\mathrm{inl}_*(f_*(p)) \cdot \mathrm{glue}(c_2) \sim \mathrm{glue}(c_1) \cdot \mathrm{inr}_*(g_*(p))$$

Proof. Put $L = \mathrm{inl}_*(f_*(p))$ and $R = \mathrm{glue}(c_1) \cdot \mathrm{inr}_*(g_*(p))$. The stored typeclass witness is $L^{-1} \cdot R \sim \mathrm{glue}(c_2)$. Congruence, associativity, inverse cancellation, and the unit law give

$$L \cdot \mathrm{glue}(c_2) \sim L \cdot (L^{-1} \cdot R) \sim (L \cdot L^{-1}) \cdot R \sim \rho \cdot R \sim R.$$

This proof is checked by the theorem `glue_natural_square_rweq` in `CompPath/PushoutCompPath.lean`. □

Rearranging gives the form used by decode:

Corollary 3.5. *For any path* $p : \mathrm{Path}(c_1, c_2)$ *in* C*:*

$$\mathrm{inl}_*(f_*(p)) \sim \mathrm{glue}(c_1) \cdot \mathrm{inr}_*(g_*(p)) \cdot \mathrm{glue}(c_2)^{-1}$$

Proof. Right-multiply Lemma 3.4 by $\mathrm{glue}(c_2)^{-1}$, reassociate, and apply $\mathrm{glue}(c_2) \cdot \mathrm{glue}(c_2)^{-1} \sim \rho$. The checked Lean theorem is `Pushout.glue_natural_rearranged_rweq`. □

3.6. **Special Cases.**

3.6.1. *Wedge Sum.* The *wedge sum* $A \vee B$ is the pushout of $A \leftarrow \mathrm{pt} \rightarrow B$:

$$A \vee B := \mathrm{Pushout}(A, B, \mathbf{1}, \lambda\_.a_0, \lambda\_.b_0)$$

This identifies the basepoints $a_0 : A$ and $b_0 : B$.

Example 3.6 (Figure-Eight as Wedge)**.** The figure-eight space is $S^1 \vee S^1$—two circles joined at a single point. The glue path identifies the basepoints of the two circles:

$$\mathrm{glue}(*) : \mathrm{Path}(\mathrm{inl}(\mathrm{base}_1), \mathrm{inr}(\mathrm{base}_2))$$

In Lean:

```
def Wedge (A : Type u) (B : Type u) (a0 : A) (b0 : B) : Type u :=
  Pushout A B PUnit' (fun _ => a0) (fun _ => b0)
```

3.6.2. *Suspension.* The *suspension* ΣA is the pushout of $\mathbf{1} \leftarrow A \rightarrow \mathbf{1}$:

$$\Sigma A := \mathrm{Pushout}(\mathbf{1}, \mathbf{1}, A, \lambda\_.*, \lambda\_.*)$$

This adds a "north pole", "south pole", and meridian paths connecting them.

In Lean:

```
def Suspension (A : Type u) : Type u :=
  Pushout PUnit' PUnit' A (fun _ => PUnit'.unit) (fun _ => PUnit
     '.unit)

noncomputable def north : Suspension A := Pushout.inl PUnit'.unit
noncomputable def south : Suspension A := Pushout.inr PUnit'.unit
noncomputable def merid (a : A) : Path north south := Pushout.
    glue a
```

3.6.3. *Bouquet of n Circles.* The *bouquet* of n circles $\bigvee_n S^1$ generalizes the wedge sum:

```
def BouquetN (n : Nat) : Type u :=
  -- n circles wedged together at a single point
  Pushout (Fin' n -> Circle) PUnit' (Fin' n)
    (fun i => Circle.circleBase)
    (fun _ => PUnit'.unit)
```

4. Free Products and Amalgamated Free Products

4.1. **Free Product Words.**

Definition 4.1 (Free Product Word)**.** A *word* in the free product $G_1 * G_2$ is an alternating sequence of elements:

```
inductive FreeProductWord (G1 : Type u) (G2 : Type u) : Type u
  | nil : FreeProductWord G1 G2
  | consLeft (x : G1) (rest : FreeProductWord G1 G2)
  | consRight (y : G2) (rest : FreeProductWord G1 G2)
```

The group operations on words are defined by concatenation and inversion with appropriate reductions.

Definition 4.2 (Word Operations)**.**

```
-- Concatenation
def concat : FreeProductWord G1 G2 -> FreeProductWord G1 G2 ->
    FreeProductWord G1 G2
  | nil, w => w
  | consLeft x rest, w => consLeft x (concat rest w)
  | consRight y rest, w => consRight y (concat rest w)

-- Inversion
def invert : FreeProductWord G1 G2 -> FreeProductWord G1 G2
  | nil => nil
  | consLeft x rest => concat (invert rest) (singleLeft (-x))
  | consRight y rest => concat (invert rest) (singleRight (-y))
```

4.2. **Amalgamated Free Product.** When G_1 and G_2 share a common subgroup H via homomorphisms $i_1 : H \to G_1$ and $i_2 : H \to G_2$, the *amalgamated free product* $G_1 *_H G_2$ identifies $i_1(h)$ with $i_2(h)$ for all $h : H$.

Definition 4.3 (Amalgamation Relation)**.**

```
inductive AmalgRelation (i1 : H -> G1) (i2 : H -> G2) :
    FreeProductWord G1 G2 -> FreeProductWord G1 G2 -> Prop
  | amalgLeftToRight (h : H) (pre suf : FreeProductWord G1 G2) :
      AmalgRelation i1 i2
        (concat pre (concat (singleLeft (i1 h)) suf))
        (concat pre (concat (singleRight (i2 h)) suf))
```

Definition 4.4 (Amalgamated Free Product)**.**

```
def AmalgEquiv (i1 : H -> G1) (i2 : H -> G2) :=
  EqvGen (AmalgRelation i1 i2)

def AmalgamatedFreeProduct (G1 G2 H : Type u)
    (i1 : H -> G1) (i2 : H -> G2) : Type u :=
  Quot (AmalgEquiv i1 i2)
```

4.3. **Full Amalgamated Free Product.** For a "canonical" target with fully reduced words, we define the *full amalgamated free product* that additionally reduces via free group laws:

```
inductive FreeGroupStep : FreeProductWord G1 G2 ->
    FreeProductWord G1 G2 -> Prop
  | cancel_left (x : G1) (pre suf : FreeProductWord G1 G2)
      (h : x + (-x) = 0) :
      FreeGroupStep (concat pre (consLeft x (consLeft (-x) suf)))
                    (concat pre suf)
  | cancel_right (y : G2) (pre suf : FreeProductWord G1 G2)
      (h : y + (-y) = 0) :
      FreeGroupStep (concat pre (consRight y (consRight (-y) suf)
         ))
                    (concat pre suf)

def FullAmalgEquiv := EqvGen (fun w1 w2 =>
  AmalgRelation i1 i2 w1 w2 \/ FreeGroupStep w1 w2)

def FullAmalgamatedFreeProduct (G1 G2 H : Type u) (i1 : H -> G1)
    (i2 : H -> G2) :=
  Quot (FullAmalgEquiv i1 i2)
```

Remark 4.5 (Amalgamation-only and full targets). `AmalgamatedFreeProduct` quotients words by the amalgamation relation alone. This relation preserves raw word length and therefore does not remove an identity letter. Consequently, the Lean theorems `hasPushoutSVKEncodeDecode_impossible` and `hasPushoutSVKDecodeAmalgBijective_impossible` prove that round-trip and bijectivity interfaces with this target cannot be inhabited. `FullAmalgamatedFreeProduct` also quotients by free-group reduction and is the target used in Theorem 5.1.

5. The Seifert-van Kampen Theorem

5.1. **Presented Seifert–van Kampen theorem.**

Theorem 5.1 (Presented Seifert–van Kampen)**.** *Let G_1, G_2 carry addition, zero, and inversion satisfying the explicit group laws recorded by* `GroupLaws`*. Let H carry the same operations, and let $i_1 : H \to G_1$ and $i_2 : H \to G_2$ preserve addition, zero, and inversion. These data are bundled by* `AmalgamationLaws`*, which records the group laws on H and the two* `PreservesGroupOps` *certificates. Form the one-vertex path presentation $\mathcal{P}(G_1, G_2; H)$ with generating edges*

$$L(x) \quad (x : G_1), \qquad R(y) \quad (y : G_2)$$

and relators

$$\begin{gathered} L(x)L(y) \sim L(x+y), \quad L(0) \sim \rho, \quad L(x)^{-1} \sim L(-x), \\ R(x)R(y) \sim R(x+y), \quad R(0) \sim \rho, \quad R(y)^{-1} \sim R(-y), \\ L(i_1 h) \sim R(i_2 h) \qquad\qquad (h : H). \end{gathered}$$

Then its presented computational fundamental group satisfies

$$\pi_1^{\mathrm{pres}}\big(\mathcal{P}(G_1, G_2; H), *\big) \cong \mathrm{FullAmalg}(G_1, G_2; H, i_1, i_2).$$

Proof. Raw paths are generated independently by `refl`, component edges, reversal, and composition. Encoding sends left and right edges to singleton words, composition to concatenation, and reversal to inverse words. Induction on generated homotopy proves that encode respects every component relator, amalgamation, congruence, and groupoid law.

Decoding replaces every word letter by its corresponding generating edge. Separate inductions show that decoding respects each free-group reduction and each amalgamation step, hence descends through `FullAmalgEquiv`. Induction on words proves encode–decode, while induction on raw paths proves decode–encode. The resulting group equivalence is Lean theorem `presentedSeifertVanKampenGroupEquiv` in `CompPath/PresentedSeifertVanKampen.lean`. □

The theorem has no encode/decode typeclass assumptions and uses no custom axioms. Its hypotheses are ordinary algebraic data printed in the declaration. The relator set is deliberately redundant: inverse relators make the structural encode proof direct even when they are derivable from multiplication, unit, and groupoid laws. Since raw paths and generated homotopy precede encode, and both round trips are proved, this redundancy does not define equality by the target normal form.

5.2. **Global-rule `PathRwQuot` schema.**

Theorem 5.2 (Conditional global-rule SVK schema)**.** *Let A, B, C be path-connected types with maps $f : C \to A$ and $g : C \to B$, and let $c_0 : C$. Assuming* `HasGlueNaturalLoopRwEq`*,* `HasPushoutSVKEncodeQuot`*,* `HasPushoutSVKDecodeEncode`*, and* `HasPushoutSVKEncodeDecodeFull`*, there is an equivalence*

$$\pi_1(\mathrm{Pushout}(A, B, C, f, g), \mathrm{inl}(f(c_0))) \simeq \mathrm{FullAmalg}\big(\pi_1(A, f(c_0)), \pi_1(B, g(c_0)); \pi_1(C, c_0)\big).$$

Here decode and its compatibility with amalgamation and free-group reduction are proved. The three encode/round-trip classes remain parameters. Because the global rewrite relation is total on parallel paths, `seifertVanKampenFullEquiv_collapsed` proves

an unconditional but degenerate instance. This schema is retained to state exactly what the global-rule API provides; it is not Theorem 5.1.

5.3. **Completed-expression normal forms.** For expression languages already equipped with certified normal forms, `ScopedCompletion` provides a shorter encode/decode interface.

Theorem 5.3 (Scoped Seifert–van Kampen)**.** *Let P_1 and P_2 be completed loop-expression presentations with normal-form types G_1 and G_2, and let $i_1 : H \to G_1$ and $i_2 : H \to G_2$. The scoped completion of alternating component expressions is equivalent to the full amalgamated product:*

$$\mathsf{ScopedSVK}(P_1, P_2; i_1, i_2) \simeq \mathsf{FullAmalg}(G_1, G_2; H, i_1, i_2).$$

Proof. Encoding maps every component expression to its normal form and then takes the full amalgamated word class. Decoding chooses a target word representative and replaces each letter by the canonical expression supplied by P_1 or P_2. Induction on words and the two component $\mathsf{encode} \circ \mathsf{decode}$ laws prove the global round trip. The other round trip is the defining completion step. Lean theorem `scopedSeifertVanKampenEquiv` in `CompPath/ScopedSeifertVanKampen.lean` checks the construction. □

This theorem remains useful for the Klein-bottle, projective-plane, lens-space, and torus expression models. For the circle and figure-eight, the stronger raw-path results are Theorem 2.10 and Theorem 5.1.

5.4. **Split Assumptions Architecture.** The SVK proof is structured around *split typeclass assumptions*, allowing maximum flexibility:

```
-- Just the encode map
class HasPushoutSVKEncodeQuot (c0 : C) where
  pushoutEncodeQuot : pi_1(Pushout A B C f g, inl (f c0)) ->
                      AmalgamatedFreeProduct (pi_1 A) (pi_1 B) (
                         pi_1 C)

-- Decode-encode round-trip (Prop)
class HasPushoutSVKDecodeEncode (c0 : C) where
  decode_encode : forall alpha, pushoutDecodeAmalg c0 (encode
     alpha) = alpha

-- Encode-decode up to FullAmalgEquiv
class HasPushoutSVKEncodeDecodeFull (c0 : C) where
  encode_decode_full : forall w, FullAmalgEquiv (encode (decode w
     )) w

-- Prop-level bijectivity (choice-based)
class HasPushoutSVKDecodeFullAmalgBijective (c0 : C) where
  bijective : Function.Bijective (pushoutDecodeFullAmalg c0)
```

5.5. **Multiple Interfaces.** The framework provides multiple equivalences depending on available assumptions:

```
-- SVK with full target (free reduction)
noncomputable def seifertVanKampenFullEquiv
    [HasPushoutSVKEncodeQuot c0] [HasGlueNaturalLoopRwEq c0]
    [HasPushoutSVKDecodeEncode c0] [HasPushoutSVKEncodeDecodeFull
        c0] :
    SimpleEquiv (pi_1(Pushout A B C f g, inl (f c0)))
                (FullAmalgamatedFreeProduct (pi_1 A) (pi_1 B) (
                   pi_1 C))

-- Choice-based full SVK (no explicit encode required)
noncomputable def
    seifertVanKampenFullEquiv_of_decodeFullAmalg_bijective
    [HasPushoutSVKDecodeFullAmalgBijective c0]
    [HasGlueNaturalLoopRwEq c0] :
    SimpleEquiv (pi_1(Pushout A B C f g, inl (f c0)))
                (FullAmalgamatedFreeProduct (pi_1 A) (pi_1 B) (
                   pi_1 C))

-- Proved for the global RwEq quotient (degenerate)
noncomputable def seifertVanKampenFullEquiv_collapsed :
    SimpleEquiv (pi_1(Pushout A B C f g, inl (f c0)))
                (FullAmalgamatedFreeProduct (pi_1 A) (pi_1 B) (
                   pi_1 C))
```

5.6. **The Encode-Decode Method.**

5.6.1. *The Decode Map.* For pushouts, the decode map converts a word to a loop:

```
noncomputable def pushoutDecode (c0 : C) :
    FreeProductWord (pi_1(A, f c0)) (pi_1(B, g c0))
    -> pi_1(Pushout A B C f g, inl (f c0))
  | .nil => Quot.mk _ (Path.refl _)
  | .consLeft alpha rest =>
      piOneMul (liftLeft alpha) (pushoutDecode c0 rest)
  | .consRight beta rest =>
      piOneMul (conjugateRight beta) (pushoutDecode c0 rest)
```

where `conjugateRight` conjugates by the glue path:

$$\beta \mapsto \mathrm{glue}(c_0) \cdot \mathrm{inr}_*(\beta) \cdot \mathrm{glue}(c_0)^{-1}$$

5.6.2. *Decode Respects Amalgamation.*

Lemma 5.4 (Decode Respects Amalgamation)**.** *For any* $\gamma : \pi_1(C, c_0)$*:*

$$\mathrm{decode}(\mathrm{consLeft}(f_*(\gamma), w)) = \mathrm{decode}(\mathrm{consRight}(g_*(\gamma), w))$$

Proof. Choose a loop representative p of γ. Corollary 3.5, specialized to $c_1 = c_2 = c_0$, identifies the left inclusion of $f_*(p)$ with the conjugated right inclusion of $g_*(p)$. Congruence under multiplication by the decoded suffix gives the displayed equality. □

In Lean:

```
theorem pushoutDecode_respects_amalg [HasGlueNaturalLoopRwEq c0]
    (gamma : pi_1(C, c0)) (rest : PushoutCode A B C f g c0) :
    pushoutDecode c0 (.consLeft (piOneFmap c0 gamma) rest) =
    pushoutDecode c0 (.consRight (piOneGmap c0 gamma) rest) := by
  induction gamma using Quot.ind with
  | _ p =>
    -- explicit association/congruence/cancellation proof
    ...
```

5.6.3. *The Encode Map.* The nondegenerate encode map is parameterized by `HasPushoutSVK EncodeQuot`; it is not a custom kernel axiom or a global instance. A code-family construction remains future work.

5.6.4. *Round-Trip Properties.* The round-trip properties establish the equivalence:

- $\mathrm{decode}(\mathrm{encode}(\alpha)) = \alpha$ (via `HasPushoutSVKDecodeEncode`)
- $\mathrm{encode}(\mathrm{decode}(w)) \sim w$ up to amalgamation and free-group reduction (via `HasPushoutSVK EncodeDecodeFull`)

6. Applications

We summarize the distinct loop-quotient, path-presentation, and model calculations formalized in Lean. Table 1 names the object being classified in each row.

Remark 6.1 (Scope of Applications). Besides the presented SVK theorem, the development contains:

- **Product formula**: $\pi_1(A \times B) \simeq \pi_1(A) \times \pi_1(B)$ (used for torus)
- **Fibration sequences**: $\pi_2(B) \to \pi_1(F) \to \pi_1(E) \to \pi_1(B)$ (used for $\mathbb{CP}^n$)
- **Presented path groupoids**: For the circle and amalgamated one-vertex path spaces
- **Presentation encode-decode**: For torus, Klein-bottle, projective-space, and lens-space expression quotients

The torus and $\mathbb{CP}^n$ are included to show the breadth of the formalization, though they do not use SVK.

Table 1: Status of the application interfaces

Space/object	Result represented in Lean	Method	Status
Presented computational fundamental groups			
$\mathcal{S}^1_{\text{pres}}$	$\pi_1^{\text{pres}} \cong \mathbb{Z}$	Raw paths and generated relators	Proved nontrivial
$\mathcal{P}(G_1, G_2; H)$	$\pi_1^{\text{pres}} \cong G_1 *_H G_2$	Presented SVK	Proved
$\mathcal{F}_{8,\text{pres}}$	$\pi_1^{\text{pres}} \cong \text{FullAmalg}(\mathbb{Z}, \mathbb{Z}; \mathbf{1})$	Presented SVK	Proved nontrivial
Global-rule loop quotients and conditional interfaces			
$A \vee B$	Full-target SVK equivalence	Pushout over $\mathbf{1}$	Conditional; collapsed form proved
$\bigvee_n S^1$	Candidate decode $F_n \to \pi_1$	n-bouquet presentation	Equivalence open
S^n $(n \geq 1)$	Global-rule $\pi_1(S^n) \simeq 1$	Universal quotient collapse	Proved; no SVK dependency
Scoped completed presentation quotients			
S^1	Scoped quotient $\simeq \mathbb{Z}$	Winding completion	Proved nontrivial
K (Klein bottle)	Scoped quotient $\simeq \mathbb{Z} \rtimes \mathbb{Z}$	Semidirect completion	Proved nontrivial
$\mathbb{RP}^2$	Scoped quotient $\simeq \mathbb{Z}_2$	Parity completion	Proved nontrivial
Σ_g, N_n	Relator syntax	Surface presentations	No presented or topological π_1 equivalence claimed
$L(p, q)$	Scoped cyclic quotient $\simeq \text{Fin}\, p$	Loop powers modulo p	Proved for $p > 0$
Other formalized objects			
T^2	Scoped product quotient $\simeq \mathbb{Z}^2$	Product of scoped circles	Proved nontrivial
$\mathbb{CP}^n$	Unit-valued presentation model	Fibration scaffold	Model only

6.1. **The Wedge Sum: Free Product of Fundamental Groups.**

Theorem 6.2 (Conditional full-target wedge schema)**.** *For pointed types* (A, a_0) *and* (B, b_0)*:*

$$\pi_1(A \vee B) \simeq \text{FullFreeProduct}(\pi_1(A), \pi_1(B)),$$

provided the full-target encode and round-trip typeclasses are supplied.

Proof. The wedge sum is the pushout $\text{Pushout}(A, B, \mathbf{1})$. Glue naturality is proved for this span. The nondegenerate equivalence still requires the full-target encode and round-trip data displayed in Theorem 5.2; under the global rewrite quotient the collapsed equivalence is unconditional. □

In Lean:

```
noncomputable def seifertVanKampenFullEquiv
    [HasPushoutSVKEncodeQuot ...]
    [HasPushoutSVKDecodeEncode ...]
    [HasPushoutSVKEncodeDecodeFull ...] :
    SimpleEquiv (pi_1(Wedge A B a0 b0, wedgeBase))
                (FullAmalgamatedFreeProduct ...)
```

6.2. **The Figure-Eight Space.**

Theorem 6.3 (Presented figure-eight fundamental group)**.**

$$\pi_1^{\text{pres}}(\mathcal{F}_8, *) \cong \mathsf{FullAmalg}(\mathbb{Z}, \mathbb{Z}; \mathbf{1}).$$

This is packaged as `figureEightPresentedPiOneGroupEquiv`, an instance of Theorem 5.1. Its two integer-labelled edge families are raw proof-relevant paths, and their homotopy is generated by the component and groupoid relators. The theorem `figureEightPresented\_nontrivial` separates the left unit generator from the identity using a left-exponent invariant.

`figureEightScopedSVKEquiv` and `figureEightProvenanceEquiv` remain lower-level normal-form and provenance equivalences. The separate declaration `figureEightPiOneEquiv` for the global-rule loop quotient is conditional on `HasWedgeSVKDecodeBijective`; no such

instance is asserted. The following code only demonstrates noncommutativity of the target word syntax:

```
def wordAB : FreeProductWord Int Int := .consLeft 1 (.consRight 1
    .nil)
def wordBA : FreeProductWord Int Int := .consRight 1 (.consLeft 1
    .nil)

theorem wordAB_ne_wordBA : wordAB != wordBA := by
  intro h; cases h  -- constructors are distinct
```

Remark 6.4 (Scope of non-commutativity). The fact that $ab \neq ba$ is a theorem about the free-product word presentation. Without a proved bridge it is not transferred to the global-rule `PathRwQuot` of the figure-eight. The presented theorem proves nontriviality; a separate commutator invariant would be required to promote this particular syntax-level inequality to a noncommutativity theorem.

6.3. **Spheres.**

Theorem 6.5 (Global-rule loop quotient of the iterated-suspension sphere)**.** *For $n \geq 1$:*

$$\pi_1(S^n) \simeq 1$$

Proof. Lean theorem `sphereN_piOne_equiv_unit` specializes the universal `pathRwQuotLoopEquivUnit`. It therefore has no SVK encode/decode dependency. □

6.4. **The Torus (Product Formula, Not SVK).**

Theorem 6.6 (Scoped torus expression quotient)**.**

$$\mathsf{TorusScopedQuotient} \simeq \mathbb{Z} \times \mathbb{Z}.$$

Remark 6.7 (Torus Construction—Not SVK). The scoped torus presentation is the product of two scoped circle-expression quotients. The global-rule loop quotient of the product carrier is contractible, and `no_torus_genuine_synthetic_bridge` rules out an equivalence between these objects. Thus this result does **not** use SVK and is not a presented or topological π_1 calculation.

$$\pi_1(A \times B, (a_0, b_0)) \simeq \pi_1(A, a_0) \times \pi_1(B, b_0)$$

No custom axiom declaration is used. A product formula is available for global-rule loop quotients, while the displayed $\mathbb{Z}^2$ equivalence is `torusScopedPiOneEquivIntProd`, obtained componentwise from scoped circle normal forms:

$$\mathrm{Path}((a_1, b_1), (a_2, b_2)) \simeq \mathrm{Path}(a_1, a_2) \times \mathrm{Path}(b_1, b_2)$$

We include the torus here to demonstrate that the formalization supports multiple π_1 calculation techniques, not just SVK.

6.5. **The Klein Bottle.**

Theorem 6.8 (Scoped Klein-bottle expression quotient)**.**

$$\mathsf{KleinBottleExprQuot} \simeq \mathbb{Z} \rtimes \mathbb{Z} = \langle a, b \mid aba^{-1}b = 1 \rangle$$

where the semidirect product has $\mathbb{Z}$ acting on $\mathbb{Z}$ by negation.

In Lean this is `kleinBottleScopedPiOneEquivIntProd`; its source is the scoped expression quotient defined in `ClassicalPresentationsScoped.lean`, not the global-rule `PathRwQuot`. The normal form is:

```
structure KleinBottlePresentation where
  -- Elements are pairs (m, n) where:
  -- m represents powers of the loop that reverses orientation
  -- n represents powers of the loop that preserves orientation
  m : Int
  n : Int

-- The relation ab = ba^{-1} induces the group operation:
-- (m1, n1) * (m2, n2) = (m1 + m2, (-1)^m2 * n1 + n2)
```

6.6. **The Real Projective Plane.**

Theorem 6.9 (Scoped projective-plane expression quotient)**.**

$$\mathsf{RP2PresentationQuotient} \simeq \mathbb{Z}_2 = \langle a \mid a^2 = 1 \rangle.$$

Proof. The theorem `realProjective2ScopedPiOneEquivZ2` classifies the parity-based scoped expression quotient in `ClassicalPresentationsScoped.lean`. No equivalence with the global-rule loop quotient is claimed. □

6.7. **Orientable Surfaces of Genus g.**

Remark 6.10 (Orientable-surface presentation status)*.*

$$\langle a_1, b_1, \ldots, a_g, b_g \mid [a_1, b_1] \cdots [a_g, b_g] = 1 \rangle$$

is represented by typed word and relator syntax. The Lean development does does not prove a presented or topological $\pi_1(\Sigma_g)$ equivalence with this presentation.

In Lean:

```
def OrientableSurfaceWord (g : Nat) : Type :=
  List (Fin' (2 * g) * Bool)  -- generator index and sign

inductive SurfaceRelation (g : Nat) :
    OrientableSurfaceWord g -> OrientableSurfaceWord g -> Prop
  | commutator_product :
      -- [a_1, b_1] ... [a_g, b_g] = 1
      SurfaceRelation g (commutatorProduct g) []
```

6.8. **Non-Orientable Surfaces.**

Remark 6.11 (Non-orientable-surface presentation status).

$$\langle a_1, \ldots, a_n \mid a_1^2 \cdots a_n^2 = 1 \rangle$$

is likewise represented as presentation syntax; no presented or topological loop-quotient equivalence is claimed.

Example 6.12 (Scoped normal forms for two special cases)**.**
- Projective-plane expressions: $\mathsf{RP2ScopedQuotient} \simeq \langle a \mid a^2 = 1 \rangle \cong \mathbb{Z}_2$.
- Klein-bottle expressions: $\mathsf{KleinScopedQuotient} \simeq \langle a, b \mid a^2 b^2 = 1 \rangle \cong \mathbb{Z} \rtimes \mathbb{Z}$.

6.9. **Lens Spaces.**

Theorem 6.13 (Scoped lens-space expression quotient)**.** *For coprime p, q:*

$$\mathsf{LensScopedQuotient}(p) \simeq \mathbb{Z}/p\mathbb{Z}.$$

Remark 6.14 (Lens Space Construction). `lensScopedPiOneEquivZp` classifies the scoped cyclic-expression quotient by its residue normal form `Fin p`. It does not derive this presentation from a quotient of all raw paths in a lens-space carrier, and it is not an SVK theorem.

6.10. **Bouquet of n Circles.**

Remark 6.15 (Bouquet status).

$$\pi_1(\bigvee_n S^1) \stackrel{?}{\simeq} F_n$$

is the expected result. Lean defines `BouquetFreeGroup n` and a candidate `decode_def`, but the full equivalence is explicitly deferred in `BouquetN.lean`.

7. Extended Features

7.1. **Higher Homotopy Groups.**

Definition 7.1 (Higher Homotopy Groups)**.** The n-th homotopy group is defined via iterated loop spaces:

$$\pi_n(A, a) := \pi_1(\Omega^{n-1}(A, a), \rho^{n-1})$$

where Ω^n denotes n-fold iteration of the loop space.

Theorem 7.2 (Eckmann-Hilton)**.** *For $n \geq 2$, $\pi_n(A, a)$ is abelian.*

Proof. The Eckmann-Hilton argument shows that when two binary operations share a common unit and satisfy an interchange law, they must coincide and be commutative. In $\Omega^2(A, a)$, both horizontal and vertical composition of 2-loops satisfy these conditions. □

In Lean:

```
def Loop2Space (A : Type u) (a : A) : Type u :=
  Derivation_2 (Path.refl a) (Path.refl a)

def PiTwo (A : Type u) (a : A) : Type u :=
  Quotient (Loop2Setoid A a)

theorem piTwo_comm (x y : PiTwo A a) : PiTwo.mul x y = PiTwo.mul
    y x := by
  induction x using Quotient.ind
  induction y using Quotient.ind
  apply Quotient.sound
  exact contractibility_3 _ _
```

7.2. **The Weak ω-Groupoid Structure.**

Theorem 7.3 (Types are Weak ω-Groupoids)**.** *Via computational paths, every type carries the structure of a weak ω-groupoid:*

- *0-cells: points of A*
- *1-cells: paths $p : \mathrm{Path}(a, b)$*
- *2-cells: derivations $\mathcal{D}_2(p, q)$ witnessing $p \sim q$*
- *3-cells: meta-derivations $\mathcal{D}_3(\alpha, \beta)$*
- *n-cells: $\mathcal{D}_n$ for all n*

In Lean:

```
-- 2-cells: derivations between paths
inductive Derivation_2 : Path a b -> Path a b -> Type u
  | refl (p : Path a b) : Derivation_2 p p
  | step (s : Step p q) : Derivation_2 p q
  | inv (d : Derivation_2 p q) : Derivation_2 q p
  | vcomp (d1 : Derivation_2 p q) (d2 : Derivation_2 q r) :
      Derivation_2 p r

-- 3-cells: derivations between 2-cells
inductive Derivation_3 : Derivation_2 p q -> Derivation_2 p q ->
    Type u
  | refl (d : Derivation_2 p q) : Derivation_3 d d
  | step (m : MetaStep_2 d1 d2) : Derivation_3 d1 d2
  | inv (m : Derivation_3 d1 d2) : Derivation_3 d2 d1
  | vcomp (m1 : Derivation_3 d1 d2) (m2 : Derivation_3 d2 d3) :
      Derivation_3 d1 d3
```

7.3. **Truncation Levels.**

Definition 7.4 (Truncation Levels)**.** Following HoTT, we define truncation levels:

- **IsContr**(A): A is contractible (has a unique point)
- **IsProp**(A): any two points are equal ("propositions")
- **IsSet**(A): any two parallel paths are equal ("sets")
- **IsGroupoid**(A): any two parallel 2-cells are equal

In Lean:

```
class IsContr (A : Type u) where
  center : A
  contr : forall a, Path center a

class IsProp (A : Type u) where
  allEq : forall (a b : A), Path a b

class IsSet (A : Type u) where
  pathEq : forall (a b : A) (p q : Path a b), RwEq p q

class IsGroupoid (A : Type u) where
  derivEq : forall (a b : A) (p q : Path a b) (d e : Derivation_2
      p q),
    Derivation_3 d e
```

7.4. **Covering Space Theory.**

Definition 7.5 (Covering Space)**.** A *covering space* of B with fiber F consists of:

- A total space E
- A projection $p : E \to B$
- A $\pi_1(B)$-action on F
- Fiber equivalence: $p^{-1}(b) \simeq F$ for each b

In Lean:

```
structure CoveringSpace (B : Type u) (F : Type u) (b0 : B) where
  totalSpace : Type u
  proj : totalSpace -> B
  fiber_equiv : forall b, Equiv (Fiber proj b) F
  deck_action : pi_1(B, b0) -> Equiv F F
  action_coherent : -- action is compatible with path lifting
```

7.5. **Fibration Theory.**

Definition 7.6 (Fibration)**.** A *fibration* $p : E \to B$ with fiber F admits a long exact sequence:

$$\cdots \to \pi_2(F) \to \pi_2(E) \to \pi_2(B) \to \pi_1(F) \to \pi_1(E) \to \pi_1(B) \to \pi_0(F) \to \pi_0(E) \to \pi_0(B).$$

`Pi0` is defined as the quotient by path-relatedness. The Lean development packages the three low-degree maps and their pointed adjacent-composition laws in `LowDegreeFibrationSequence`; the constructor is `lowDegreeFibrationSequence` in `Fibration.lean`. If F, E, B are path-connected, their π_0 types are subsingletons and the customary terminal 1's may be used as shorthand.

7.6. **Eilenberg–MacLane benchmark.** Classically, S^1 is $K(\mathbb{Z}, 1)$. In the Lean development, `circleScopedPiOneEquivInt` establishes the $\mathbb{Z}$ normal form for the scoped circle-expression quotient. This result is not transported to the global-rule `PathRwQuot`; consequently, no theorem in this paper identifies the formalized circle carrier with an Eilenberg–MacLane space.

7.7. **Path Simplification Tactics.** The formalization includes convenience tactics for path reasoning:

```
-- Basic simplification using groupoid laws
path_simp    -- refl . p ~ p, p . refl ~ p, etc.

-- Full automation (~25 simp lemmas)
path_auto    -- handles most standalone RwEq goals

-- Specialized tactics
path_rfl           -- close p ~ p
path_canon         -- normalize to canonical form
path_cancel_left   -- p^-1 . p ~ refl
path_cancel_right  -- p . p^-1 ~ refl
path_congr_left h  -- apply hypothesis on right
path_congr_right h -- apply hypothesis on left
```

To quantify their use, `paper/svk/check_stats.py` performs a comment-stripped lexical count. None of the 19,931 declarations introduced with `theorem`/`lemma` directly invokes a `path_*` tactic. Across all 53,378 declarations counted (including `def`, `instance`, and `example`), 11 invoke one directly (approximately 0.021%). In the nineteen modules used by this paper, one of 945 declarations does so (0.106%). Thus the tactics are small convenience wrappers; the SVK proofs are predominantly explicit applications of named `RwEq` lemmas, congruence, and quotient induction.

8. The Lean 4 Formalization

8.1. **Kernel axioms and parameterized assumptions.** Lean 4 does not natively support higher-inductive types. This development therefore does *not* postulate HIT constructors: pushouts are ordinary quotients, and the nontrivial loop calculations are presentation syntax. The invariant script `scripts/check_invariants.py` checks the entire source tree after stripping comments.

Definition 8.1 (Kernel Axiom)**.** A *custom kernel axiom* is a Lean `axiom` declaration in the project source. There are none in the source tree. Declarations may still depend on standard Lean kernel principles such as quotient soundness, propositional extensionality, or classical choice; these dependencies can be inspected declaration-by-declaration with `#print axioms`.

Definition 8.2 (Typeclass Assumption)**.** A *typeclass assumption* is a hypothesis provided via Lean's typeclass mechanism. These are **not** kernel axioms—they are parameters that users instantiate. We use these for:

(1) **Naturality**: `HasGlueNaturalLoopRwEq` (glue commutes with paths)

(2) **Nondegenerate encode/decode data**: `HasPushoutSVKEncodeQuot`, `HasPushoutSVKDecodeEncode`, and `HasPushoutSVKEncodeDecodeFull`

Remark 8.3 (Axioms versus assumptions). The distinction is crucial:

- **Custom kernel axioms**: zero in the source tree.
- **Typeclass assumptions** are user-provided hypotheses. They appear in theorem statements as `[HasFoo]` parameters, making dependencies explicit.

For example, `seifertVanKampenFullEquiv` prints all four required typeclasses in its declaration. By contrast, `seifertVanKampenFullEquiv_collapsed` has no encode assumptions and records the degenerate theorem forced by the global rewrite quotient.

Remark 8.4 (Status of encode/decode data). There are no global axiom instances for encode/decode. An amalgamation-only encode/decode round trip is impossible because that relation preserves word length while decode removes identity letters. The nondegenerate full-target encode classes remain explicit open parameters. The global-rule loop quotient itself is contractible on every carrier, as proved by `pathRwQuot_loop_contractible`.

8.2. **When to Use Each SVK Assumption.** The split typeclass architecture allows users to provide minimal hypotheses:

Assumption	**When to Use**
`HasGlueNaturalLoopRwEq`	Required by the general decode lift; proved for wedges and other named cases. Under the global $\sim$ relation, `hasGlueNaturalLoopRwEq_collapsed` discharges it generally.
`HasPushoutSVKEncodeQuot`	When you have an explicit encode function.
`HasPushoutSVKDecodeEncode`	Standard encode-decode round-trip.
`HasPushoutSVKEncodeDecodeFull`	Round-trip modulo amalgamation and free-group reduction.
`HasPushoutSVKDecodeFullAmalgBijective`	Choice-based full-target interface when only decode bijectivity is available.

8.3. **Architecture.** The Lean 4 formalization is organized as follows:

Module	Content
`Path/Basic/Core.lean`	Path type, refl, symm, trans
`Path/Basic/Congruence.lean`	congrArg, transport
`Path/Rewrite/Step.lean`	Single-step rewrites (50+ rules)
`Path/Rewrite/RwEq.lean`	Rewrite equality
`Path/Rewrite/ScopedCompletion.lean`	Presentation-specific completion relations
`Path/Rewrite/PathTactic.lean`	Automated tactics
`Path/Homotopy/Loops.lean`	Loop spaces
`Path/Homotopy/FundamentalGroup.lean`	π_1 definition
`Path/Homotopy/PresentedFundamentalGroup.lean`	Raw presented paths, generated homotopy, and π_1^{pres}
`Path/Homotopy/HigherHomotopy.lean`	π_n for $n \geq 2$
`Path/CompPath/PushoutCompPath.lean`	Quotient pushout and glue naturality
`Path/CompPath/PushoutPaths.lean`	Word quotients, decode, conditional full SVK
`Path/CompPath/PushoutSVKInstances.lean`	Obstructions and collapsed full SVK
`Path/CompPath/ScopedSeifertVanKampen.lean`	Scoped SVK encode/decode equivalence
`Path/CompPath/PresentedSeifertVanKampen.lean`	Presented SVK and figure-eight theorem
`Path/CompPath/CirclePresented.lean`	Presented circle fundamental group $\simeq \mathbb{Z}$
`Path/CompPath/CircleScoped.lean`	Circle and torus scoped completions
`Path/CompPath/ClassicalPresentationsScoped.lean`	Klein, projective, and lens completions
`Path/CompPath/FigureEight.lean`	Conditional and provenance interfaces
`Path/CompPath/SuspensionDeep.lean`	Iterated spheres under the global relation
`Path/Homotopy/Fibration.lean`	π_1 sequence and π_0 tail
`Path/TypeTheory/QuotientPathInduction.lean`	Universal global-loop collapse
`Path/TypeTheory/MetadataRepair.lean`	Global/scoped no-bridge results
`Path/OmegaGroupoid.lean`	Weak ω-groupoid structure

8.4. **Kernel-axiom inventory.** The source tree contains **zero custom `axiom` declarations**. Scoped presentation equivalences are ordinary proved definitions; open nondegenerate global-rule SVK data remain parameters rather than global axioms.

8.5. **Statistics.**

Metric	Value
Lean lines in the 19-module formalization core	11,796
Modules in the formalization core	19
Custom `axiom` declarations	0
`sorry`/`admit` occurrences after comment stripping	0/0
Reproduction command	`python3 paper/svk/check_stats.py`

8.6. **Assumption Manifest.** Theorem 5.1 has no encode/decode assumptions: its algebraic `GroupLaws` and `AmalgamationLaws` arguments are ordinary explicit structures; `AmalgamationLaws` includes the two operation-preservation certificates. For reference, the following table records only the parameters of the separate global-rule schema, Theorem 5.2.

Typeclass	Role	Status
`HasGlueNaturalLoopRwEq`	Glue naturality	Proved for named cases; generally derivable in the collapsed system
`HasPushoutSVKEncodeQuot`	Nondegenerate encode map	Open parameter
`HasPushoutSVKDecodeEncode`	Decode–encode	Open parameter
`HasPushoutSVKEncodeDecodeFull`	Full encode–decode	Open parameter
`HasPushoutSVKDecodeFullAmalgBijective`	Alternative full-decode bijectivity	Open choice-based parameter

Proved: the presented path groupoid construction, both presented SVK maps and round trips, circle and figure-eight specializations, the global decode map, respect for amalgamation and free-group reduction, impossibility of the amalgamation-only target, and the collapsed global-rule equivalence. There are no opt-in axiom instances.

9. Library Highlights

Beyond the SVK framework, the library includes extended features for advanced homotopy-theoretic constructions:

9.1. **Higher Homotopy Groups.** The higher homotopy groups $\pi_n(A, a)$ for $n \geq 2$ are defined via iterated loop spaces:

$$\pi_n(A, a) := \pi_1(\Omega^{n-1}(A, a), \rho)$$

Key results include:

- Abelianness of π_2 via Eckmann-Hilton (`HigherHomotopy.lean`)
- Long exact sequence of a fibration (`Fibration.lean`)

9.2. **Weak ω-Groupoid Structure.** Types are shown to form weak ω-groupoids via the derivation tower:

- 1-cells: paths $\mathrm{Path}(a, b)$
- 2-cells: derivations $\mathcal{D}_2(p, q)$ witnessing $p \sim q$
- 3-cells: derivations $\mathcal{D}_3(d_1, d_2)$ between derivations
- And so on, with truncation at level 3 (`OmegaGroupoid.lean`)

9.3. **Truncation Levels.** The library formalizes HoTT-style truncation levels:

- `IsContr`: Contractible types (all points equal)
- `IsProp`: Propositions (all proofs equal)
- `IsSet`: Sets (all paths equal)
- `IsGroupoid`: Groupoids (all 2-paths equal)

Module: `Truncation.lean`

9.4. **Covering Spaces.** Covering space theory with π_1-actions:

- Fiber transport action (`CoveringSpace.lean`)
- Galois correspondence framework (`CoveringClassification.lean`)

9.5. **Additional Features.**

- **Eilenberg–MacLane models**: typed model structures and classical benchmark statements (`EilenbergMacLane.lean`); no carrier-level $S^1 = K(\mathbb{Z}, 1)$ theorem is claimed here
- **Mayer-Vietoris**: Abelianized SVK sequence (`MayerVietoris.lean`)
- **Path tactics**: Automation via `path_simp`, `path_auto`, `path_normalize` (`PathTactic.lean`)

10. Conclusion

We have presented a *modular SVK framework* within the computational paths setting, demonstrating that:

(1) Pushouts are implemented via Lean's quotient types without custom axioms, and the glue-naturality square and rearranged corollary have explicit Lean proofs.
(2) Proof-relevant generator graphs and named relators define presented path groupoids independently of normalization. Their fundamental groups are automorphism groups of basepoints.
(3) The presented SVK encode and decode maps are both constructed, both round trips are proved, and the resulting fundamental group is equivalent to the full amalgamated free product. The circle and figure-eight specializations are nontrivial.
(4) Presented fundamental groups, global-rule loop quotients, conditional schemas, and completed expression quotients are distinguished throughout.
(5) The nineteen modules used by the paper comprise 11,796 lines, and the repository contains no `sorry`, `admit`, or custom `axiom` declaration.

Table 2: Comparison with Standard HoTT

Aspect	**Standard HoTT**	**Computational Paths**
Path equality	Identity type (Id)	Syntactic $(\sim)^*$
Coherence	Abstract existence	Explicit rewrite derivations
SVK assumptions	Monolithic	Explicit relator/algebra data
Encode function	Code family via J	Structural recursion on raw paths
ω-groupoid	Higher identity types	Derivation tower

*Decidable only relative to an explicitly supplied `CompleteStepSystem`; see §2.4.

10.1. Comparison with HoTT Approaches.

10.2. Limitations.

(1) **Topological realization**: Theorem 5.1 computes the fundamental group of a presented computational path groupoid. No theorem yet identifies that groupoid with the ordinary topological fundamental groupoid of a geometric realization.
(2) **Quotient collapse**: Although raw paths retain `List (Step A)`, the global $\sim$ relation is total on parallel paths. Global-rule loop quotients are therefore contractible. Presented homotopy avoids this collapse by admitting only the named relators and groupoid laws.
(3) **Global-rule schema**: The separate theorem 5.2 still has explicit encode and round-trip parameters.

10.3. **Future Work.**

(1) **Cubical type theory comparison**: Relate computational paths to cubical approaches.
(2) **Geometric realization**: Construct realizations of the presented path groupoids and compare π_1^{pres} with the topological fundamental group.
(3) **Covering space classification**: Prove the Galois correspondence between covering spaces and subgroups of π_1.
(4) **Higher Hopf fibrations**: Quaternionic ($S^3 \to S^7 \to S^4$) and octonionic ($S^7 \to S^{15} \to S^8$) sequences.
(5) **3-manifold classification**: Seifert fibered spaces, graph manifolds.
(6) **Homology theory**: Extend from π_1 to full homology groups.

References

[1] Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. Cubical type theory: A constructive interpretation of the univalence axiom. *Mathematical Structures in Computer Science*, 28(7):1228–1269, 2018.

[2] Ruy J. G. B. de Queiroz, Anjolina G. de Oliveira, and Arthur F. Ramos. Propositional equality, identity types, and direct computational paths. *South American Journal of Logic*, 2(2):245–296, 2016.

[3] Ruy J. G. B. de Queiroz and Dov M. Gabbay. Equality in labelled deductive systems and the functional interpretation of propositional equality. In *Proceedings of the 9th Amsterdam Colloquium*, pages 547–565, 1994.

[4] Tiago M. L. de Veras, Arthur F. Ramos, Ruy J. G. B. de Queiroz, and Anjolina G. de Oliveira. On the calculation of fundamental groups in homotopy type theory by means of computational paths. *arXiv preprint arXiv:1804.01413*, 2018.

[5] Tiago M. L. de Veras, Arthur F. Ramos, Ruy J. G. B. de Queiroz, and Anjolina G. de Oliveira. A topological application of labelled natural deduction. *South American Journal of Logic*, 2023.

[6] Tiago M. L. de Veras, Arthur F. Ramos, Ruy J. G. B. de Queiroz, and Anjolina G. de Oliveira. Computational paths – a weak groupoid. *Journal of Logic and Computation*, 35(5):exad071, 2023.

[7] Allen Hatcher. *Algebraic Topology*. Cambridge University Press, 2002.

[8] Kuen-Bang Hou (Favonia) and Michael Shulman. A mechanization of the Blakers-Massey connectivity theorem in homotopy type theory. In *LICS*, pages 565–574. ACM, 2016.

[9] The HoTT–Agda contributors. Homotopy type theory in Agda: `VanKampen`. `https://github.com/HoTT/HoTT-Agda`, accessed July 2026.

[10] The Mathlib contributors. Fundamental groupoid and fundamental group of a topological space. `https://leanprover-community.github.io/mathlib4_docs/Mathlib/AlgebraicTopology/FundamentalGroupoid/Basic.html`, accessed July 2026.

[11] Nicolai Kraus and Jakob von Raumer. A rewriting coherence theorem with applications in homotopy type theory. *Mathematical Structures in Computer Science*, 32(7):982–1014, 2022.

[12] Daniel R. Licata and Michael Shulman. Calculating the fundamental group of the circle in homotopy type theory. In *LICS*, pages 223–232. IEEE, 2013.

[13] J. Peter May. *A Concise Course in Algebraic Topology*. University of Chicago Press, 1999.

[14] Arthur F. Ramos, Ruy J. G. B. de Queiroz, and Anjolina G. de Oliveira. On the identity type as the type of computational paths. *Logic Journal of the IGPL*, 25(4):562–584, 2017.

[15] Arthur F. Ramos, Ruy J. G. B. de Queiroz, Anjolina G. de Oliveira, and Tiago M. L. de Veras. Explicit computational paths. *South American Journal of Logic*, 4(2):441–484, 2018.

[16] Arthur F. Ramos. *Explicit Computational Paths in Type Theory*. PhD thesis, Universidade Federal de Pernambuco, 2018.

[17] Arthur F. Ramos, Tiago M. L. de Veras, Ruy J. G. B. de Queiroz, and Anjolina G. de Oliveira. Computational paths in Lean. `https://github.com/Arthur742Ramos/ComputationalPathsLean`, 2025.

[18] Herbert Seifert. Konstruktion dreidimensionaler geschlossener Räume. *Berichte Sächs. Akad. Wiss. Leipzig*, 83:26–66, 1931.

[19] Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013.

[20] Egbert R. van Kampen. On the connection between the fundamental groups of some related spaces. *American Journal of Mathematics*, 55(1):261–267, 1933.

Appendix A. Application Inventory

The following table provides the exact Lean names, module paths, objects, and dependency status for the headline results. Presented fundamental groups, global-rule `PathRwQuot` fibers, and completed expression quotients are listed as distinct objects.

Object/result	**Lean name**	**Module**	**Status**
Presented circle fundamental group $\simeq \mathbb{Z}$	`CirclePresented.piOneEquivInt`	`CirclePresented`	Proved nontrivial
General presented SVK group equivalence	`presentedSeifertVanKampenGroupEquiv`	`PresentedSeifertVanKampen`	Proved
Presented figure-eight fundamental group	`figureEightPresentedPiOneGroupEquiv`	`PresentedSeifertVanKampen`	Proved nontrivial
Every global-rule loop quotient $\simeq 1$	`pathRwQuotLoopEquivUnit`	`QuotientPathInduction`	Proved
Iterated sphere global-rule loop quotient $\simeq 1$	`sphereN_piOne_equiv_unit`	`SuspensionDeep`	Proved; no SVK
Scoped circle expression quotient $\simeq \mathbb{Z}$	`circleScopedPiOneEquivInt`	`CircleScoped`	Proved nontrivial
Scoped torus expression quotient $\simeq \mathbb{Z}^2$	`torusScopedPiOneEquivIntProd`	`CircleScoped`	Proved nontrivial
Scoped figure-eight SVK equivalence	`figureEightScopedSVKEquiv`	`ScopedSeifertVanKampen`	Proved nontrivial
General scoped SVK equivalence	`scopedSeifertVanKampenEquiv`	`ScopedSeifertVanKampen`	Proved
Nondegenerate full SVK schema	`seifertVanKampenFullEquiv`	`PushoutPaths`	Four displayed classes
Collapsed full SVK schema	`seifertVanKampenFullEquiv_collapsed`	`PushoutSVKInstances`	Proved, degenerate
Amalgamation-only target obstruction	`hasPushoutSVKEncodeDecode_impossible`	`PushoutSVKInstances`	Proved
Scoped Klein presentation $\simeq \mathbb{Z} \rtimes \mathbb{Z}$	`kleinBottleScopedPiOneEquivIntProd`	`ClassicalPresentationsScoped`	Proved nontrivial
Scoped $\mathbb{RP}^2$ presentation $\simeq \mathbb{Z}_2$	`realProjective2ScopedPiOneEquivZ2`	`ClassicalPresentationsScoped`	Proved nontrivial
Scoped lens presentation $\simeq \mathrm{Fin}\, p$	`lensScopedPiOneEquivZp`	`ClassicalPresentationsScoped`	Proved for $p > 0$
Bouquet candidate decode	`BouquetN.decode_def`	`BouquetN`	Equivalence open
π_0 fibration tail	`lowDegreeFibrationSequence`	`Fibration`	Proved maps/laws

There are no opt-in axiom imports in the source tree.